Towards Multi-Turn Dialog Systems for Industrial Asset Operations and Maintenance

Rujing Li, Yitong Bai, Chengrui Li, Rui Li
Columbia University
New York, NY, USA
{rl3641, yb2636, cl4750, rl3586}@columbia.edu

GitHub: Multi-Turn-AssetOps; Evaluation **WandB Dashboard:** Experiment-Tracking

***Abstract*—Industrial asset operations and maintenance question answering is inherently multi-turn, iterative, and highly dependent on external tool invocation. However, conventional Plan-Execute single-agent architectures exhibit clear limitations in maintaining cross-turn context, and reusing intermediate results. In this paper, we present a multi-turn dialog system designed for industrial scenarios based on a supervisor-specialist multi-agent architecture. To alleviate tool-invocation bottlenecks, the system incorporates structured artifact reuse, dynamic replanning, and parallel tool execution. Evaluation results show that our system achieves better response quality compared with the baseline, with planning effectiveness increasing by 41.6% and task completion improving by 30.4%. System profiling further shows that cross-turn artifact reuse effectively reduces redundant tool invocation, decreasing the tool-time share from 47.3% to 26.3% and making turns 2–5 approximately 4.2× faster than the first turn.**

***Index Terms*—Multi-turn dialog systems, industrial asset operations, multi-agent architecture, tool-augmented LLMs.**

I. Introduction

A. Background and Motivation

Driven by the emergence of Industry 4.0 technology, continuous monitoring and automated control of physical assets are increasingly employed to improve industrial operations and maintenance (O&M), thereby facilitating the adoption of predictive maintenance and more energy-efficient processes [1]. However, making effective decisions regarding O&M tasks frequently involves reasoning across diverse sources, such as databases, equipment knowledge, operation contexts, etc. In real-world diagnostic settings, users rarely pose a single-shot question; rather, they often iterate through their queries by posing additional questions based on prior responses.

This creates a need for multi-turn dialogue systems which can preserve context, reuse intermediary results, and coordinate multiple analytical capabilities. In the field of industry O&M, tool invocation is often the dominant source of latency and cost. The continuous querying of the same time series data, work order records, alerts, and failure-code mappings not only leads to higher latency but also to instability and inefficiency. Therefore, an effective industrial diagnostic assistant must be able to remember useful intermediate artifacts, and adapt its reasoning process as new information becomes available.

B. Problem Statement

The central problem we focus on is that of developing a robust, efficient and reliable multi-turn dialogue system for industrial O&M. Existing single-agent plan-execute system [2] is not well suited for tool-centric industrial diagnosis because it is benchmark-oriented, treating tasks as independent instances without mechanisms for cross-turn memory and artifact reuse.

Specifically, the baseline architecture has three major limitations. Firstly, it does not provide enough planning capabilities necessary for multi-turn, multi-step diagnostic tasks requiring coordination of time-series, work-orders, alerts and maintenance. Secondly, the architecture is unreliable due to frequent tool invocation failures when the system hallucinates tool parameters or fails to recover from the failed executions. Lastly, intermediate results cannot be reused effectively, leading to multiple invocations and higher costs in multi-turn tasks.

To overcome these problems and better meet the demands of practical multi-turn industrial O&M tasks, this paper explores the shift from single-turn to multi-turn interactions and from the single agent-based architecture to a multi-agent system that includes a supervisor and multiple specialist agents [3].

C. Objectives and Scope

Our objective is to design, implement, and assess a multi-turn dialogue system for industrial O&M that both improves diagnostic quality and reduces unnecessary tool usage. The assessment consists of two parts: evaluation and profiling.

The evaluation focuses on response quality, comparing our system with baseline methods using seven metrics that cover both subjective and objective metrics. Specifically, subjective metrics are assessed by LLM-as-judge [4], while objective metrics are computed through rule-based validation.

In addition to response quality, we conduct profiling analysis of our system. The scope is centered on system-level optimization, and we focus on latency and token cost during end-to-end task execution, including the time spent on LLM generation, tool invocation, and different dialogue turns. We also analyze the number of LLM calls and token consumption to understand how multi-agent coordination and artifact reuse affect computational cost and overall system efficiency.

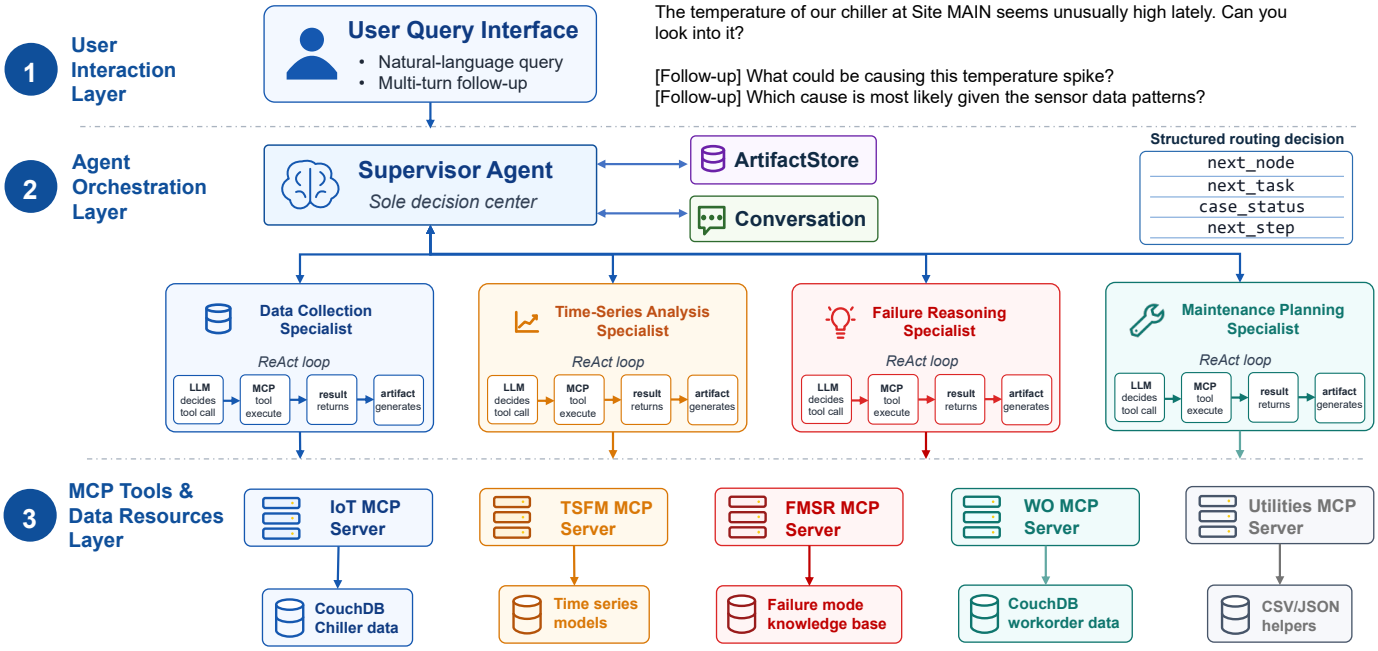


Fig. 1. Overview of our supervisor-specialist multi-agent system for industrial asset operations. The system contains three layers: user interaction layer, agent orchestration layer and MCP tools & data resources layer. Users can input natural language query, and ask follow-up questions.

II. Literature Review

A. Domain-Specific Language Agents for Industrial O&M

Industry 4.0 maintenance leverages real-time sensor data, maintenance records, and domain knowledge [5]. However, existing works primarily concentrate on the prediction tasks, including fault detection and prognosis. In contrast, the actual end-to-end maintenance workflows remain less explored. ReAct shows that interleaving reasoning steps and environmental actions enhances decision-making while alleviating hallucination for tool-based tasks [6]. For industrial applications, ReActXen applies this paradigm to natural-language access over SCADA and IoT data, introducing IoTBench for industrial sensor-query evaluation [1]. AssetOpsBench further provides the context of industrial asset operations and maintenance as a benchmark task for agents to reason with perception and control [2]. These studies suggest domain-specific language agent applications, but they do not address how agents maintain context or reuse tools in multi-turn interactions.

B. Memory and Multi-Turn Interaction

Industrial multi-turn O&M differs from single-turn question answering because later user turns often require previous data retrieval, intermediate results, and outputs from tools. To address this problem, the approach of retrieval-augmented generation has been proposed, which combines parameterized priors with non-parameterized external memory [7]. Multi-turn O&M dialogues, however, not only require retrieval of relevant documents but also reusable execution artifacts. MemGPT strives to coordinate the virtual context across memory layers so that agents can maintain extended interactions beyond the context window [8]. Reflexion records verbal feedback in an episodic memory buffer to support future decision-making [9]. Such approaches illustrate the important role of memory in agent behavior; however, they are not designed for tool-centric industrial scenarios, where latency is dominated by repeated tool invocation.

C. Multi-Agent Collaboration

Complex O&M tasks need coordinated efforts of various capabilities, including time-series analysis, alert interpretation, data retrieval, and maintenance suggestion. It limits the scalability of single-agent architectures and motivates multi-agent decomposition. General LLM multi-agent frameworks have explored several coordination patterns [10]. Recently, enterprise-oriented work has studied supervisor-specialist hierarchies, where a supervisor plans, delegates, and aggregates outputs from specialized agents [3]. These findings have encouraged the integration of multi-agent collaboration within end-to-end industrial O&M discussed in this paper.

III. Methodology

A. System Overview

The proposed system adopts a supervisor-specialist multi-agent framework for multi-turn industrial O&M question answering. As shown in Fig. 1, there are three layers to this multi-agent framework: user interaction layer, agent orchestration layer, and MCP tools and data resources layer. The user interaction layer receives the initial question and any follow-up questions, while maintaining dialog state and reusable artifacts from previous turns. At the agent orchestration layer, there is a supervisor agent and several specialist agents. The supervisor interprets the intention of the user, figures out what evidence has been acquired, breaks down the task into subtasks, and then assigns them to the corresponding specialist. The tools

and data resources layer is where external capabilities are made accessible via MCP servers such as IoT sensor retrieval, time-series forecasting, anomaly detection, work-order lookup, event and alert retrieval, and failure-code mapping.

In our implementation, the data collection specialist retrieves asset-level operational evidence, the time-series specialist performs forecasting and anomaly scoring, the failure-reasoning specialist links abnormal observations with possible failure modes, and the maintenance-planning specialist converts the diagnostic evidence into recommended actions. This decomposition follows the supervisor-specialist pattern discussed in prior multi-agent work, where a central coordinator delegates specialized subtasks and aggregates their outputs [3].

Each specialist returns a structured artifact rather than only a free-form text answer. An artifact records the asset identifier, time range, invoked tools, key observations, intermediate results, and any assumptions or confidence signals. The supervisor then synthesizes these artifacts into the final response for the current turn and stores them for later turns. This design is particularly important for industrial O&M dialogs because follow-up questions often refer implicitly to prior evidence, such as "the same chiller," "that anomaly," or "the previous maintenance record." By grounding later reasoning in stored artifacts, the system can preserve context without repeatedly retrieving the same time-series or work-order data.

B. Optimization Techniques

To address the limitations of the plan-execute baseline, we introduce three system-level optimization techniques. These techniques target the main bottlenecks observed in tool-centric industrial scenarios: repeated external tool calls, fragile tool-argument construction, and inefficient sequential execution.

1) Cross-Turn Artifact Reuse: Since industrial diagnostic conversations often revisit the same asset, time window, or maintenance evidence, many tool calls in a naive system are redundant. Our system stores the structured outputs generated by specialists. Before calling a new tool, the supervisor checks whether an existing artifact already satisfies the current request. If the user asks a follow-up question that can be answered from previous evidence, the supervisor reuses the artifact directly; if the follow-up requires additional information, only the missing portion is retrieved.

2) Dynamic Specialist Routing and Replanning: In the Plan-Execute baseline, a single agent generates a plan and follows it in a mostly linear manner. This strategy is fragile when the initial plan omits necessary evidence, selects the wrong tool, or constructs invalid tool arguments. In contrast, our supervisor re-evaluates the workflow at each turn and after each specialist result. Routing decisions depend on the current user intent, available artifacts, missing evidence, and tool execution status.

3) Parallel Tool Execution: Some diagnostic subtasks require multiple independent data sources, such as retrieving sensor history, recent alerts, historical work orders, and failure-code mappings for the same asset. When these calls do not depend on each other, executing them sequentially increases wall-clock latency unnecessarily. In the advanced supervisor-specialist architecture, independent MCP tool calls within a specialist can be launched concurrently and then normalized into a single structured artifact after all results return.

C. Evaluation Pipeline

The evaluation component is intended to assess whether a multi-turn industrial dialog agent can not only generate fluent responses but also plan effectively, call the right tools, follow tool schemas, execute successfully, and recover from failures. For this evaluation, we consider two kinds of input to the pipeline: the ground truth for the scenario, and the rollout log files generated by each candidate system. The ground truth file contains the expected task intent, assets, target evidence, and execution trajectory, while the rollout log files contain information about the user input, the system response, tool calls and their arguments, and their status.

The pipeline first normalizes the rollout data into a standardized dialog format. Normalizing the rollout data into a standardized format is necessary since the rollout data might vary between different architectures. In particular, our implementation supports rollout data in two possible formats, event stream or QA history, and then normalizes them into `DialogRecord` class. It contains information on the dialog level (such as metadata), turn level (such as turn information), and tool-call level (such as tool-call traces).

This evaluation then splits into three branches that work concurrently. The first branch involves a subjective evaluation that scores the effectiveness of planning, the quality of tool utilization, and whether tasks are completed. The second branch assesses tool-call validity based on schema matching and log parsing. In this case, the branch ensures that each tool call corresponds to a valid tool name, that the argument schema matches the expected values, and that the process succeeds. Lastly, the third branch evaluates the recovery behavior by testing the ability to continue towards a final answer despite any retries, repairs, and replans.

We use seven metrics to characterize both response quality and execution reliability, as shown in Table I. The first three metrics are subjective dialog-level scores in $[0, 1]$ and are macro-averaged across dialogs, while the next three metrics are automatic call-level scores and are micro-averaged over tool calls. The last metric measures whether the system can recover from intermediate failures such as invalid arguments, failed tool invocations, or inadequate evidence.

D. Profiling Pipeline

The profiling module tests efficiency at the system level with respect to the performance of three architectures: the Plan-Execute baseline architecture, the Supervisor-Specialist architecture (SS), and an advanced version of the Supervisor-Specialist architecture (SSA) which allows for parallel MCP tool execution within each specialist module. Each of the three architecture modules undergoes profiling using the exact same set of 16 multi-turn industrial dialogs.

Profiling is done in three tiers. The first tier captures all calls made to the LLM API using a callback function called `CustomLogger` which is attached in LiteLLM, including the name of the model, prompt tokens, completion tokens,

TABLE I
EVALUATION METRICS FOR MULTI-TURN INDUSTRIAL DIALOG SYSTEM.

Metric	Level	Aggregation and Description
Planning Effectiveness	Dialog-level subjective metric	Macro-averaged across dialogs. Evaluates whether the agent decomposes the task properly, maintains goal alignment, and re-plans when necessary.
Tool Usage Quality	Dialog-level subjective metric	Macro-averaged across dialogs. Evaluates whether the agent selects appropriate tools, constructs valid arguments, and uses context effectively across turns.
Task Completion	Dialog-level subjective metric	Macro-averaged across dialogs. Measures whether the final response successfully addresses the user's diagnosis or maintenance planning needs.
Tool Name Validity	Call-level objective metric	Micro-averaged over all extracted tool calls. Measures the percentage of tool calls whose tool names exist in the available tool set, indicating whether the agent avoids hallucinating unavailable tools.
Schema Compliance	Call-level objective metric	Micro-averaged over valid-name tool calls. Measures the percentage of valid-name tool calls whose arguments satisfy the required schema and structured input constraints.
Execution Success	Call-level objective metric	Micro-averaged over all extracted tool calls. Measures the percentage of tool calls that execute successfully without runtime failure.
Recovery Success Rate	Dialog-level recovery metric	Computed over dialogs where a recovery action was taken. Measures the percentage of such dialogs that eventually complete successfully after retrying, correcting invalid arguments, or re-planning.

TABLE II
EVALUATION RESULTS COMPARING THE PLAN-EXECUTE BASELINE WITH THE SUPERVISOR-SPECIALIST ARCHITECTURE.

Architecture	Plan. Eff.	Tool Qual.	Task Comp.	Name Val.	Schema Comp.	Exec. Succ.	Recovery SR
Plan-Execute	0.559	0.731	0.710	1.000	0.680	0.648	0.850
Supervisor-Specialist	0.791	0.850	0.925	1.000	0.900	0.856	1.000

and latency for each completion. The second tier captures all MCP tool invocation functions within the agents, recording the server name, tool name, latency, and status of each call. The third tier captures all CouchDB `find()` calls within the IoT, work-order, and vibration MCP servers, recording the type of query, latency, number of documents found, and whether there were any errors.

The following are recorded as dialog-level metrics and shown on the WandB dashboard [11]: wall time, a three-way latency decomposition involving LLM time, tool time, and routing time (the residual after subtracting the first two from wall time), tokens consumed, number of LLM API calls, and per server tool latency. Turn-level metrics track turn duration, turn success, and length of the turn output with a unique turn index across all dialogs to observe how the per-turn costs change as dialogs deepen.

In addition to dialog-level metrics, tracing via LangSmith [12] can be enabled if `LANGCHAIN_API_KEY` exists. This captures per-call prompts and responses, while the supervisor branches capture node-level LangGraph traces that record route choices and dispatched specialties.

IV. EXPERIMENTAL RESULTS

A. Evaluation Results

Table II provides the summary of the evaluation leaderboard for the Plan-Execute approach and the Supervisor-Specialist framework. The findings from the evaluation demonstrate improvement in both the quality of the response on a subjective measure and the reliability of tool calling on an objective measure. For instance, Planning Effectiveness increases from 0.559 to 0.791, Tool Usage Quality increases from 0.731 to 0.850, and Task Completion increases from 0.710 to 0.925.

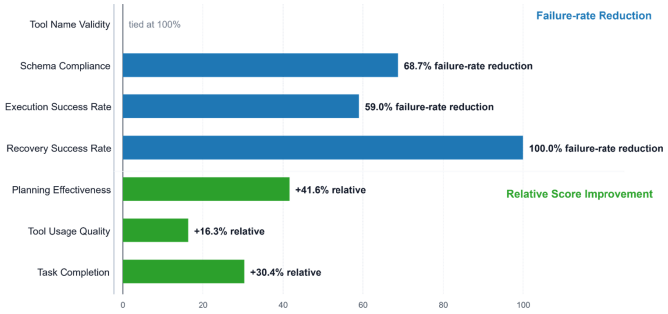


Fig. 2. Per-metric improvement of the Supervisor-Specialist system over the Plan-Execute baseline. Blue bars represent failure-rate reduction for objective execution metrics, green bars represent relative improvement in subjective scores, and gray labels indicate tied metrics.

The objective metrics reveal a higher degree of reliability in execution. There is a tie between the two systems in terms of tool-name validity at 1.000, which means that both systems remain within the tool boundary. There is, however, an increase in schema compliance from 0.680 to 0.900, and an increase in execution success from 0.648 to 0.856. This reflects a decrease of 68.7% in schema failure and a decrease of 59.0% in execution failure.

Fig. 2 also shows a breakdown of the improvement over the baseline in relative terms. Subjectively, the improvements are in planning (41.6% better), tool usage (16.3% better), and task completion (30.4% better). Reliability-wise, the highest relative gains are seen in schema compliance and execution success, confirming that fewer invalid or failed tool calls are made. These gains are consistent with the design principle underlying the construction of the evaluation pipeline: a multi-turn industrial agent must be evaluated both on its end answers

and on planning, tool invocation, schema compliance, and successful execution.

In summary, based on the evaluations, the Supervisor-Specialist architecture seems more appropriate for tool-centric multi-turn industrial conversations than the Plan-Execute baseline architecture. Specifically, because specialist agents handle domain-specific tasks, fewer tool-call errors occur, thereby increasing the quality of generated answers.

B. Profiling Results

1) Run-Level Summary: Table III summarizes completion statistics and resource consumption across the three architectures over 16 dialogs.

TABLE III
RUN-LEVEL PROFILING SUMMARY

Metric	Plan-Execute	SS	SSA
Total wall time (min)	83.9	**65.2**	73.3
Total tokens consumed	**2.55M**	3.32M	3.62M
Total LLM API calls	841	941	**751**

The baseline consumes the fewest tokens. SS completes fastest at 65.2 minutes. SSA incurs the highest token cost, reflecting the overhead of parallel tool result aggregation.

2) Wall-Time Latency Decomposition: Fig. 3 reports the three-way latency breakdown, averaged across all 16 dialogs.

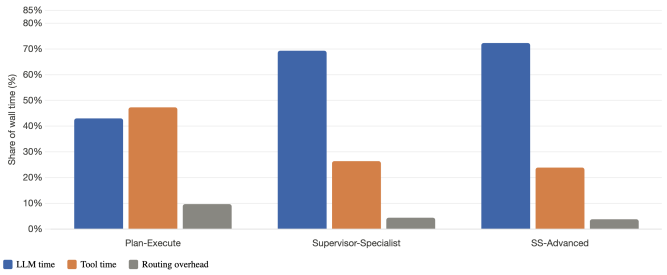


Fig. 3. Wall-time latency decomposition by architecture.

Tool invocation emerges as the major bottleneck in the baseline Plan-Execute architecture, constituting 47.3% of the wall time (average 186.6 s per dialogue). The two supervisor designs lower the tool percentage to a similar value of about 25%, with SS and SSA yielding almost equal tool fractions (26.4% and 23.9%) even though the latter introduces parallel execution. It is evident that supervisor routing rather than tool parallelism is the major factor for tool efficiency. As the tool overhead is reduced, the LLM API latency becomes the bottleneck factor, contributing 69-72% to the wall time in the two supervisor architectures.

3) TSFM as the Critical Tool Bottleneck: The single source of architectural differences in tool latencies is found in the per-server analysis of time-series foundation model (TSFM) server. On average, the Plan-Execute baseline is 159.5s per dialog on TSFM server, compared with 4.3 times longer than the SS (37.4s) and the SSA (36.2s) and, at the same time, 85.5% of the entire tool time spent by the baseline in this server alone, sometimes lasting for as long as 495s. This strongly suggests that the Plan-Execute reasoning strategy issues redundant or unbounded time-series queries when multi-step reasoning diverges, whereas the supervisor routes to the TSFM specialist more precisely and terminates once sufficient data is obtained. IoT server latency is equally decreased to 12.9s in SS from 24.9s due to removal of duplicated sensor history queries. Work-order, utilities, and FMSR servers are unaffected by the architecture difference in absolute latency but their rising proportional share in supervisor architectures is a denominator effect from TSFM's reduction.

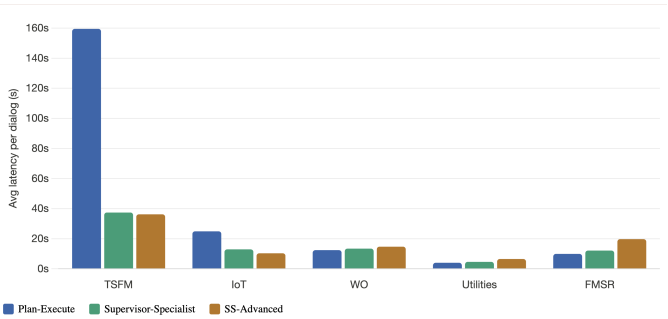


Fig. 4. Average MCP tool server latency per dialog.

4) Context Growth and Token Cost: SSA has a context-bloat issue caused by parallel tool combination. As soon as all parallel tool invocations return, the outputs are concatenated to generate the input to the next LLM invocation. The average number of tokens used per invocation in SSA is 4,825 (59% higher than that in the baseline, 3,036), while the 95th percentile of the number of tokens used per invocation is 14,272 (more than double that in the baseline, 6,361). For example, in SSA Dialog 3, the total number of tokens used is 1,438,115, which is 3.5× that in the second-largest case from any architecture. This issue accounts for the failure of SSA in handling timeouts: as the context size increases, each LLM invocation takes longer to compute, which gives rise to the longest single invocation time of 122.7 s (compared to 44.1 s in the baseline) and a slow-call rate (>10 s) of 6.9% versus 2.0% for the baseline.

5) Per-Turn Cost Distribution: Table IV and Fig. 5 show mean turn durations by position within each dialog.

TABLE IV
MEAN TURN DURATION BY POSITION (SECONDS)

Turn	Plan-Execute	SS	SSA
1	69.3	145.4	169.7
2	89.6	18.4	14.4
3	90.9	37.5	55.5
4	59.6	26.2	61.8
5	105.7	184.0	26.9
Turn 2–5 avg	83.4	**34.3**	40.9

The Supervisor-Specialist architecture front-loads cost into turn 1 (SS 145 s, SSA 170 s) due to LangGraph initialization and the cold-start routing dispatch, but subsequent turns drop to 34.3 s and 40.9 s respectively as the graph warms and artifacts are reused, which is a 4.2× speedup consistent with the abstract claim. The baseline takes the opposite pattern:

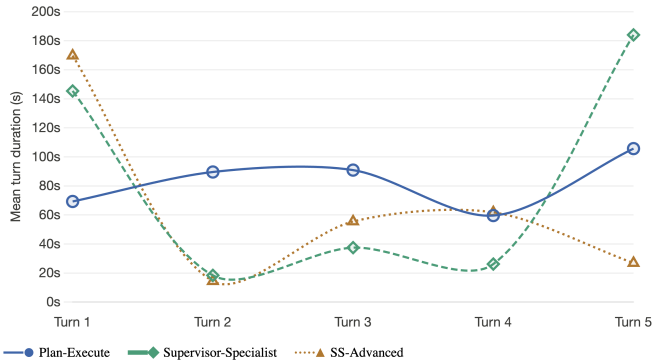


Fig. 5. Mean turn duration by position within a dialog.

turn 1 is cheapest (69.3 s) while later turns average 83.4 s, with response length growing monotonically from 1,100 to 6,700 characters (6×), reflecting unbounded context accumulation. SS and SSA maintain stable response lengths of 2,700–3,200 characters throughout.

V. Discussion

A. Interpretation of Results

The results show that the supervisor-specialist architecture improves multi-turn industrial O&M dialogue mainly by enabling structured task decomposition and cross-turn artifact reuse. Compared with the plan-execute baseline, the proposed system produces better planning, tool usage, and task completion, while also reducing schema and execution failures. The profiling results further suggest that the key efficiency gain comes from avoiding redundant tool calls, especially repeated time-series forecasting queries. Although the supervisor introduces more overhead in the first turn, this cost is amortized in later turns because previously generated evidence can be reused for follow-up questions.

B. Limitations

Several limitations remain in the current study. First, the evaluation is based on 16 benchmark dialogs and a limited set of chiller assets, so the results may not fully represent the diversity of real industrial sites, failure modes, and user behaviors. Second, the system relies on available benchmark data and predefined MCP tool interfaces, which may be cleaner and more structured than production O&M environments. Third, the parallel tool execution setting increases prompt size and token consumption, which shifts the bottleneck from tool latency to LLM generation latency and prevents SSA from consistently outperforming the simpler SS architecture.

C. Future Work

Future work should focus on improving both system efficiency and evaluation coverage. On the system side, parallelism should move from low-level tool calls to parallel specialist execution, combined with evidence summarization, pruning, and selective artifact retrieval to reduce prompt bloat. On the evaluation side, future benchmarks should include larger and more realistic industrial datasets, longer dialogs, and memory-specific metrics such as artifact reuse rate, memory recall accuracy, redundant tool-call reduction, and cross-turn consistency. These improvements would help determine whether the system can generalize from controlled benchmark scenarios to real industrial O&M workflows.

VI. Conclusion

This paper presents a multi-turn dialog system for industrial asset operations and maintenance based on a Supervisor-Specialist multi-agent architecture. The system addresses key limitations of a Plan-Execute single-agent baseline, including weak cross-turn context management, fragile tool invocation, and redundant retrieval of expensive operational evidence. By decomposing diagnostic workflows into specialist agents and storing their outputs as reusable artifacts, the proposed architecture supports memory-aware replanning and more reliable tool use across follow-up questions.

The evaluation results show that the Supervisor-Specialist architecture improves response quality and execution reliability. Compared with the baseline, it increases planning effectiveness, tool usage quality, and task completion, while also improving schema compliance and execution success. The profiling results show that the main efficiency gain comes from reducing redundant tool usage, especially repeated TSFM calls. Although the supervisor architecture incurs higher first-turn overhead, later turns become substantially faster because previously generated artifacts can be reused.

The results also show that parallel tool execution is not automatically beneficial. In our experiments, SSA reduces some tool-side latency but increases token consumption and LLM generation time due to larger aggregated contexts. This finding suggests that future system optimization should focus not only on parallelism, but also on compact evidence representation, selective memory retrieval, and efficient inter-agent communication.

Overall, this work demonstrates that multi-agent decomposition and structured artifact reuse are effective design principles for tool-centric industrial O&M dialog systems. While broader datasets and more realistic deployment conditions are still needed, the proposed approach provides a promising foundation for building robust, efficient, and context-aware assistants for industrial asset operations.

References

[1] J. T. Rayfield, S. Lin, N. Zhou, and D. C. Patel, "ReAct meets industrial IoT: Language agents for data access," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics, 2025, pp. 364–382. [Online]. Available: https://aclanthology.org/2025.emnlp-industry.24/

[2] IBM, "AssetOpsBench: Ai agents for industrial asset operations and maintenance," https://github.com/IBM/AssetOpsBench, 2025.

[3] R. Shu, N. Das, M. Yuan, M. Sunkara, and Y. Zhang, "Towards effective genai multi-agent collaboration: Design and evaluation for enterprise applications," 2024. [Online]. Available: https://arxiv.org/abs/2412.05449

[4] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems*, vol. 36. Curran Associates, Inc., 2023, pp. 46 595–46 623. [Online]. Available: https://dl.acm.org/doi/10.5555/3666122.3668142

[5] T. Zonta, C. A. da Costa, R. da Rosa Righi, M. J. de Lima, E. S. da Trindade, and G. P. Li, "Predictive maintenance in the industry 4.0: A systematic literature review," *Computers & Industrial Engineering*, vol. 150, p. 106889, 2020.

[6] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://arxiv.org/abs/2210.03629

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive nlp tasks," in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.

[8] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards llms as operating systems," 2023. [Online]. Available: https://arxiv.org/abs/2310.08560

[9] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[10] LangChain, "Benchmarking multi-agent architectures," https://blog.langchain.com/benchmarking-multi-agent-architectures, 2024.

[11] Weights & Biases, "Weights & Biases: The AI developer platform," https://wandb.ai, 2020.

[12] LangChain, "LangSmith: Observability and evaluation for LLM applications," https://smith.langchain.com, 2023.

APPENDIX

In this project, AI tools such as ChatGPT, Codex, and Claude were used as support tools.

The AI tools were used in the following ways:

1) **Background reading and clarification.**
   During initial research, we utilized AI tools to understand public resources such as the description of the Hugging Face dataset, AssetOpsBench source code, and examples of implementation for agent systems. This was done to enhance our understanding of the systems involved, relevant terminologies, and design patterns.
2) **Code debugging and assistance.**
   All codes used in this project were created, reviewed, improved, and tested by our team. The codex were used to assist in debugging by helping us to understand error messages, debugging directions, refactoring suggestions, and optimizations.
3) **Reading, translation, and language polishing.**
   While reading related papers and documentation, we used AI tools for clarification of technical language. During report writing, AI tools were used only for partial translation, grammar correction, prose polishing, and improving academic wording for the content already drafted by the team.